GUIA TÉCNICO

Keras Functional e arquiteturas flexíveis

Capítulo 08/20 · Fundamentos, experiência reproduzível e interpretação

PROPÓSITO E OBJETIVOS

- Compreender grafos acíclicos de camadas, múltiplas entradas e ramificações.
- Relacionar a técnica à sua função de perda, representação e protocolo de avaliação.
- Explicar limitações e formular um teste reprodutível.

MAPA DE ESTUDO

1. Fundamentação do método
2. Expressões e hipóteses
3. Experimento essencial executável
4. Laboratório aplicado e limitações
5. Exercício e verificações

01 FUNDAMENTAÇÃO CONCEITUAL

A API Functional conecta tensores simbólicos através de operações em grafo; é útil para entradas múltiplas, junções e saídas distintas. A correspondência entre entradas e rótulos deve ser inequívoca.

Fonte histórica de apoio: notas do curso original, consultadas apenas como referência de contexto; as explicações e exemplos desta edição são independentes.

Questões críticas: quais pressupostos são necessários? O que a função aprende? Qual métrica indica utilidade real? Que tipo de vazamento ou má interpretação invalidaria a conclusão?

02 MATEMÁTICA E DIMENSÕES

Expressão esquemática: f(x_a,x_b)=g([h_a(x_a),h_b(x_b)])

A expressão acima resume a relação central. Para a derivação, identifique dimensões, hipóteses e operações; não interprete notação compacta como uma implementação completa.

LEITURA CRÍTICA DO CAPÍTULO

Modelos Functional possuem grafo acíclico explícito e permitem entradas paralelas. A fusão NumPy é apenas explicação das dimensões.

Procedimento: execute a próxima célula com o núcleo declarado em requirements.txt, confira os assert e interprete os resultados. O experimento é independente dos exemplos anteriores.

03 EXPERIMENTO ESSENCIAL

Código extraído do notebook revisado. Execute a célula completa no JupyterLab; as quebras visuais abaixo não substituem o arquivo executável.

```
import numpy as np
rng=np.random.default_rng(42)
a=rng.normal(size=(6,2));b=rng.normal(size=(6,3))
merged=np.concatenate([a,b],axis=1)
print('Duas entradas:',a.shape,b.shape,'concatenação:',merged.shape)
assert merged.shape==(6,5)
```

INTERPRETAÇÃO

Compare o resultado obtido com a formulação matemática. Confirme shapes, escala e hipótese de modelagem. Um experimento sintético valida uma operação, não substitui avaliação em dados reais. Ao treinar modelos, separe dados e ajuste transformações apenas no treino; preserve um conjunto final de teste.

04 LABORATÓRIO APLICADO

Laboratório aplicado revisado — Dois caminhos de entrada e fusão de representações

Modelos Functional possuem grafo acíclico explícito e permitem entradas paralelas. A fusão NumPy é apenas explicação das dimensões.

Procedimento: execute a próxima célula com o núcleo declarado em requirements.txt, confira os assert e interprete os resultados. O experimento é independente dos exemplos anteriores.

```
import numpy as np
rng=np.random.default_rng(8)
a=rng.normal(size=(16,3));b=rng.normal(size=(16,2))
Wa=rng.normal(size=(3,4));Wb=rng.normal(size=(2,4))
ha=np.maximum(a@Wa,0);hb=np.maximum(b@Wb,0)
concat=np.concatenate([ha,hb],axis=1)
assert concat.shape==(16,8)
print('Entrada A:',a.shape,'Entrada B:',b.shape,'Fusão:',concat.shape)
print('Uma API Functional conecta duas entradas e une os tensores em um grafo.')
```

Fusão NumPy explica formas e fluxos. Não equivale à execução de um modelo Keras Functional; conferir o laboratório opcional de Keras separadamente.

Checklist: conferir entradas e saídas, sementes, inexistência de uso dos dados de teste durante ajuste e distinção entre uma demonstração sintética e desempenho em aplicações reais.

05 EXERCÍCIO E SOLUÇÃO COMENTADA

Quando preferir Functional a Sequential?

Resposta: Quando houver conexões não lineares entre camadas, ramificações, múltiplas entradas ou saídas.

LIMITAÇÕES E BOAS PRÁTICAS

- Confundir ajuste no treino com generalização para dados não vistos.
- Reportar somente uma métrica ou uma execução aleatória.
- Ignorar shape, tipo de variável e convenção do backend.
- Executar exemplos com pesos baixados sem registrar versão, licença e procedência.
- Confundir demonstração matemática com validação de um modelo de produção.

VERIFICAÇÃO LOCAL

Execute Reiniciar kernel e executar todas as células. Registre versões do Python e das dependências, confira shapes e métricas e diferencie os laboratórios opcionais dos exemplos NumPy/scikit-learn.

Validação posterior (13/11/2026): notebooks 14 a 20 executados com TensorFlow CPU; opcionais 01 a 13 não integralmente revalidados.

REFERÊNCIAS TÉCNICAS

NumPy: numpy.org/doc · scikit-learn: scikit-learn.org/stable/user_guide.html · Keras: keras.io/guides · TensorFlow: tensorflow.org/guide

REVISÃO DIDÁTICA · CAPÍTULO 08

Functional: grafos de computação e duas entradas

1. Conceito, escopo e interpretação

A API Functional modela um grafo acíclico de tensores simbólicos. Diferentemente de simplesmente concatenar arrays NumPy, criar o grafo Keras implica definir Input, conectar camadas e instanciar Model(inputs=...,outputs=...).

2. Formalização e exemplo resolvido

a=Input(shape=(3,)); b=Input(shape=(2,)); h1=Dense(6,relu)(a); h2=Dense(2,relu)(b); z=Concatenate()([h1,h2]); out=Dense(1)(z); model=Model([a,b],out). A fusão tem 6+2=8 atributos.

3. Relação com o notebook e experimento

Repita a fusão NumPy do notebook (matriz Bx6 concatenada a Bx2). No laboratório Keras opcional, apresente model.summary(), treine com as duas entradas separadas e valide a métrica no conjunto de teste.

4. Limitações e erros frequentes

A demonstração de shape NumPy não constitui treinamento da Functional API. Um modelo com múltiplas entradas deve preservar correspondência entre linhas de cada entrada e rótulo.

5. Exercício comentado

Se os ramos produzem (B,6) e (B,2), qual shape da fusão ao longo do último eixo? (B,8). O que muda com eixos de batch incompatíveis? A operação falha.

6. Ampliação conceitual do guia

A Functional API define tensores simbólicos via keras.Input, conecta camadas e constrói keras.Model(inputs=[...], outputs=...). Duas entradas podem alimentar ramos distintos; Concatenate une as representações no eixo de features, desde que outras dimensões sejam compatíveis.

7. Orientação de estudo e validação

O laboratório NumPy do notebook demonstra a forma da fusão, não treina um keras.Model. Um laboratório Keras opcional deve criar dois Input, dois ramos Dense, Concatenate e um Dense final; compilar e avaliar com dados separados, relatando que não foi executado nesta geração.

8. Exercício adicional de fixação

Exercício: ramos com shape (B,6) e (B,2) unidos por Concatenate(axis=-1) resultam em qual shape? Resposta: (B,8).

Verificação: execute o notebook homônimo no ambiente apropriado; registre versões, shapes, sementes e métricas. Laboratórios Keras opcionais não foram executados durante esta revisão documental.

Abrir notebook do capítulo no repositório

LABORATÓRIO COMPLEMENTAR · CAPÍTULO 8

Laboratório Keras Functional — duas entradas

Implementação ilustrativa / formalização

```
import numpy as np
from keras import Input, Model, layers
rng = np.random.default_rng(42)
a = rng.normal(size=(160, 3)).astype('float32')
b = rng.normal(size=(160, 2)).astype('float32')
y = (a[:, :1] + b[:, :1]).astype('float32')
x1 = Input(shape=(3,), name='grupo_a')
x2 = Input(shape=(2,), name='grupo_b')
z1 = layers.Dense(6,activation='relu')(x1)
z2 = layers.Dense(4,activation='relu')(x2)
z = layers.Concatenate()([z1,z2])
out = layers.Dense(1)(z)
model = Model([x1,x2],out)
model.compile(optimizer='adam',loss='mse')
model.fit([a[:120],b[:120]],y[:120],epochs=8,verbose=0)
print(model.evaluate([a[120:],b[120:]],y[120:],verbose=0))
```

Interpretação e limites

Aqui a concatenação ocorre dentro de um grafo Keras. Observe 6+4=10 features na fusão; ambas as entradas e o alvo devem manter a ordem de amostras. A separação é ilustrativa e não substitui experimentos com validação adequada.

Como verificar

Execute o código no ambiente indicado pelo notebook. Se depender de Keras/TensorFlow, registre versões e resultados reais; esta revisão não afirma execução do laboratório.

Referência: notebook do mesmo capítulo em 07_deep_learning (GitHub).